
GoTalks 28.05.2026
Go Project Structures
Breakdown of most common ones
Go
CONCURRENCY
package main
CONCURRENCY
1.22
imprt

How to reach us

meetup.com/Golang-ZG

@golangzg

github.com/golanghr/golangzg

@golangzg.bsky.social

invite.slack.golangbridge.org

About me

Matej Baćo

- Gopher since ~2015
- ~20 years in IT engineering / development
- I'm a bit strange

The question

Go has no official project structure.

So... what do I do?

What we'll cover

- 8 layout approaches using same example app
- From flat structure to hexagonal architecture
- Popular layouts from 2014 to today (2026)
- **My** take on prominent common Go project layouts
- In the wild there are mixes
- Some I did not encounter for sure
- If you have favorite layout I did not list, show us!

The example application

An office library web app:

- Users browse books
- Users borrow and return books
- Users write reviews

Domain type User

```go
// User represents a library member
type User struct {
    ID    int
    Name  string
    Email string
}

type UserService interface {
    User(id int) (User, error)
    Users() ([]User, error)
    CreateUser(user User) error
}
```

Domain type Book

```
type Book struct {
    ID       int
    Title    string
    Author   string
    Borrowed *User
}

type BookService interface {
    Book(id int) (Book, error)
    Books() ([]Book, error)
    Borrow(bookID int, userID int) error
    Return(bookID int) error
}
```

Domain type Review

```go
type Review struct {
    ID     int
    BookID int
    UserID int
    Text   string
    Rating int
}

type ReviewService interface {
    Reviews(bookID int) ([]Review, error)
    CreateReview(review Review) error
}
```

Layout #1 - Flat / No Structure

Also known as "Simple Layout"

by Peter Bourgon, "Go in Production" (talk at GopherCon 2014)

Flat - Directory structure

```
.
├── main.go
├── user.go
├── book.go
├── review.go
├── handler.go
└── go.mod
```

Everything in `package main`.

Flat - main.go

```go
package main

import (
    "database/sql"
    "log"
    "net/http"
    _ "github.com/lib/pq"
)

func main() {
    db, err := sql.Open("postgres", "connection string")
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()

    us := &userService{DB: db}
    bs := &bookService{DB: db}
    rs := &reviewService{DB: db}

    h := handler{Users: us, Books: bs, Reviews: rs}
    log.Fatal(http.ListenAndServe(":8080", h))
}
```

Flat - When it works

- Prototypes and hackathons
- Small CLI tools (under ~10 files)
- Learning projects
- When you genuinely don't know the domain yet

Flat - When it breaks

- More than ~10 files - hard to find things
- Multiple developers - merge conflicts everywhere
- Need for test isolation - can't swap implementations
- Reusable logic - everything is `package main`

Layout #2 BJ Standard Package Layout

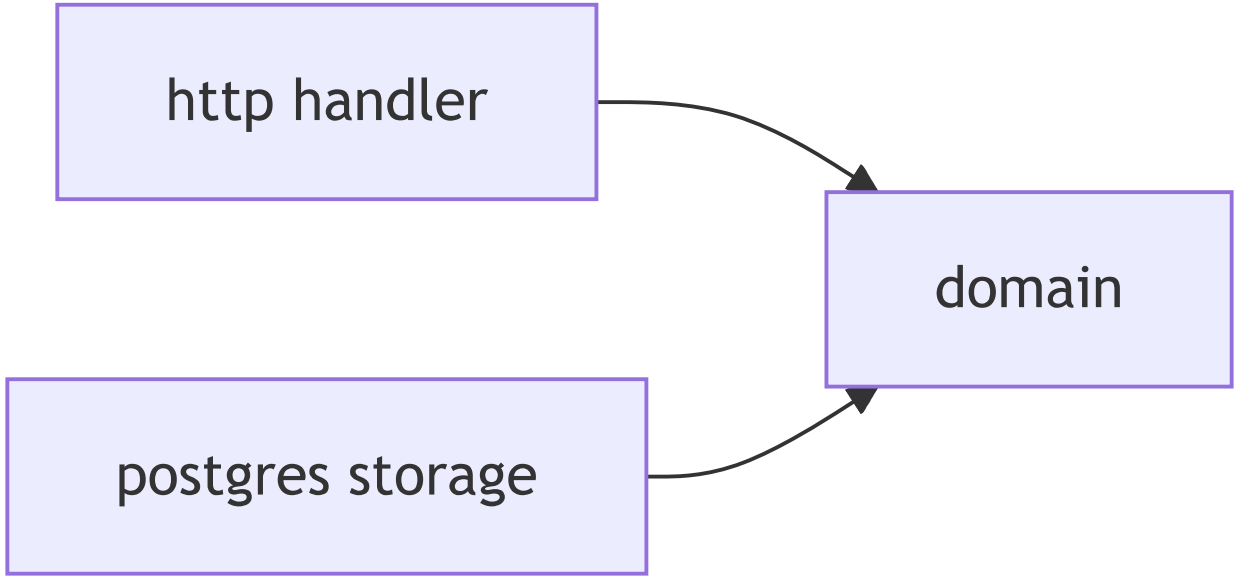
Ben Johnson, "Standard Package Layout" (2016)

The single most influential Go project layout.

BJ - Directory structure

```
.
├── cmd/
│   └── server/
│       └── main.go
├── user.go             # domain types & interfaces (root package)
├── book.go
├── review.go
├── http/               # HTTP transport
│   ├── handler.go
│   └── server.go
├── postgres/           # storage implementation
│   ├── user_service.go
│   ├── book_service.go
│   └── review_service.go
└── mock/               # test doubles
    ├── user.go
    ├── book.go
    └── review.go
```

BJ - Key idea: Dependency Inversion



BJ - Key idea: Dependency Inversion

Both `http/` and `postgres/` depend on the root package (the domain). **The root package depends on NOTHING.**

BJ - Domain in root package

```go
// user.go - root package, no external imports
package library

type User struct {
    ID    int
    Name  string
    Email string
}

type UserService interface {
    User(id int) (User, error)
    Users() ([]User, error)
    CreateUser(user User) error
}
```

BJ - Implementation in leaf packages

```go
// postgres/user_service.go
package postgres

import (
    "database/sql"
    "github.com/myorg/library"
)

type UserService struct {
    DB *sql.DB
}

func (us *UserService) User(id int) (library.User, error) {
    // SQL query here
    return library.User{}, nil
}
```

BJ - Composition root

```go
// cmd/server/main.go - wires everything together
package main

import (
    "log"
    "net/http"
    "github.com/myorg/library/http"
    "github.com/myorg/library/postgres"
)

func main() {
    db, _ := postgres.Open("connection")
    us := &postgres.UserService{DB: db}
    bs := &postgres.BookService{DB: db}

    h := &http.Handler{
        UserService: us,
        BookService: bs,
    }
    log.Fatal(http.ListenAndServe(":8080", h))
}
```

BJ - When it works

- Small-to-medium web services
- When you want testability
- When you want domain separated from "infrastructure"
- The "next step up" from flat layout

BJ - When it breaks

- Root package can get large
- No `internal/` - all packages importable externally and need to be supported
- Multiple domains compete in the same root package
- `mock/` package grows unbounded

BJ - Historical significance

- The root package **IS** the domain
- Root package is named by what the project and domain is e.g. `library`
- The core insight: **"domain in root, implementations in leaf packages, main wires them"** appears in hexagonal, clean, and feature-based layouts
- First introduced in "Structuring Applications in Go" (2014), then refined in "Standard Package Layout" (2016)

Layout #3 WK - Package-Oriented Design

William Kennedy from Ardan Labs "Package-Oriented Design" (2017)

Kennedy - Directory structure

```
.
├── cmd/
│   └── library/
│       └── main.go
├── internal/
│   ├── platform/
│   │   ├── database/
│   │   │   └── postgres.go
│   │   └── http/
│   │       └── server.go
│   ├── users/
│   │   ├── model.go
│   │   └── service.go
│   ├── books/
│   │   ├── model.go
│   │   └── service.go
│   └── reviews/
│       ├── model.go
│       └── service.go
└── go.mod
```

Kennedy - Three layers of packages

1. **Domain packages** (`internal/users/`, `internal/books/`, `internal/reviews/`)
   - Business logic, models, service interfaces
   - The core of the application
2. **Platform packages** (`internal/platform/`)
   - Foundational, reusable infrastructure - database, HTTP server, middleware
   - NOT business-specific
3. **Composition root** (`cmd/library/`)
   - Wires domain and platform together

Kennedy - Domain package

```go
// internal/users/model.go
package users

type User struct {
    ID    int
    Name  string
    Email string
}
```

```go
// internal/users/service.go
package users

import "database/sql"

type Service struct {
    DB *sql.DB
}

func (s *Service) User(id int) (User, error) {
    return User{}, nil
}
```

Kennedy - Platform & composition root

```go
// internal/platform/database/postgres.go
package database

import "database/sql"

func Open(connString string) (*sql.DB, error) {
    return sql.Open("postgres", connString)
}
```

```go
// cmd/library/main.go
package main

import (
    "log"
    "os"
    "github.com/example/internal/platform/database"
    "github.com/example/internal/users"
)

func main() {
    db, _ := database.Open(os.Getenv("DB"))
    userService := users.Service{DB: db}
    // ... wire handlers, start server
}
```

Kennedy - When it works

- Medium-to-large applications
- In multi team with multi repo setup `internal/` enforces boundaries
- Enterprise/Corporate Go

Kennedy - When it breaks

- `internal/platform/` can become a "junk drawer"
- Domain packages **may** e.g. directly import `database/sql`, nothing is preventing "infrastructure" to leak into domain
- No prescribed way to **share** common libraries between repos
- Overkill for small services

Layout #4 PB - Complex Layout

Peter Bourgon "Go Best Practices 2016"

The `pkg/` convention - everything importable goes under `pkg/`.

Bourgon - Directory structure

```
.
├── cmd/
│   └── server/
│       └── main.go
├── pkg/
│   ├── http/
│   │   ├── handler.go
│   │   └── server.go
│   ├── library/
│   │   ├── book.go
│   │   ├── user.go
│   │   └── review.go
│   └── postgres/
│       ├── user_service.go
│       ├── book_service.go
│       └── review_service.go
└── go.mod
```

The pkg/ debate

"`pkg/` doesn't make much sense with Go modules. In the GOPATH era, `pkg/` separated pre-compiled packages. With modules, this distinction doesn't exist."

- Eli Bendersky (2019)

"Don't use a `pkg/` directory. It's a convention from pre-modules Go
that doesn't provide any benefit today."

- Reddit r/golang (2024)

"`pkg/` is useful in monorepos to distinguish between internal

Bourgon - Verdict

- Popular in monorepos where multiple binaries share packages
- `pkg/` **is semantically meaningless with Go modules** - single-binary apps don't need it
- Peter Bourgon himself has moved toward simpler layouts

Layout #5 - golang-standards/project-layout

Community repo on GitHub (~56k stars)

Not an official Go standard despite the name.

Standard Layout - The full menu

```
.
├── cmd/           # Main applications (each subdir = one binary)
├── internal/      # Private application code
├── pkg/           # "Reusable packages" (debated)
├── api/           # API protocol definitions
├── web/           # Web assets
├── configs/       # Configuration files
├── scripts/       # Build/deploy scripts
├── test/          # Additional integration tests
├── docs/          # Documentation
├── build/         # Build output
└── deployments/   # Docker, K8s manifests
```

Standard Layout - The critical perspective

"It is not an official Go standard, despite the name, so beginners can mistake it for endorsed guidance."

- Hacker News (2021)

"The layout can encourage cargo-cult programming: people copy the structure because it looks professional, not because they understand the tradeoffs."

- kau.sh (2024)

Standard Layout - Controversy

Problems:

- Misleading name - "golang-standards" implies official endorsement
- `pkg/` is outdated with Go modules
- Over-engineered for most projects
- Cargo cult adoption - copied without understanding

But also:

- 56k stars means many people find it useful
- Individual directory conventions are fine on their own
- Best treated as a **menu**, not a blueprint
- When you DO need `/api/`, `/deployments/`, etc. - this shows where they go

The 2020–2026 shift

New approaches emerged since 2019 when first version of this talk was done.

Some prominent ones:

- Feature-Based / Domain-Driven layout
- Hexagonal inspired layout

Timeline

```
2016  Ben Johnson, Peter Bourgon, William Kennedy publish approaches
 |
2018  golang-standards/project-layout becomes popular
 |    Go 1.11 introduces modules (preview)
 |
2019  First version of this talk
 |
2021  Go 1.16 modules are enabled by default
 |    GOPATH is dead, pkg/ loses meaning
 |
2022  Go 1.18 generics, workspaces (go.work)
 |    Multi-module monorepos become practical
 |
2024  Go 1.22 - stdlib HTTP enhanced routing patterns Eliminates 3rd-party routers
 |    Feature-based / domain-driven layouts gain momentum
 |
2026  This talk!
```

What changed?

1. **Go Modules (2019)** - `pkg/` largely unnecessary; packages already namespaced by module path
2. `go.work` **(2022)** - multi-module monorepos practical - new layout patterns
3. **Go 1.22 ServeMux (2024)** - no more 3rd-party routers - simpler handler organization
4. **Community maturity** - 10 years of seeing what works and what doesn't
5. **Hexagonal / DDD patterns** - proven in Java/C#, now adapted (carefully) for Go

Layout #6 Feature-Based / Domain-Driven

Multiple sources - Alex Edwards (2023), community consensus (2024–2026)

Organize by FEATURE, not by LAYER.

Feature-Based - Layer vs Feature

Layer-based (avoid):

```
internal/
├── handlers/       # all HTTP handlers mixed together
├── services/       # all business logic mixed together
├── models/         # all data types mixed together
└── repositories/   # all database code mixed together
```

Feature-based (prefer):

```
internal/
├── user/           # everything about users
│   ├── handler.go
│   ├── service.go
│   ├── repository.go
│   └── model.go
├── book/           # everything about books
│   ├── handler.go
│   ├── service.go
│   ├── repository.go
│   └── model.go
└── platform/       # shared infrastructure only
    ├── database/
    └── middleware/
```

Feature-Based - Directory structure

```
.
├── cmd/
│   └── server/
│       └── main.go
├── internal/
│   ├── user/
│   │   ├── handler.go
│   │   ├── service.go
│   │   ├── repository.go
│   │   └── model.go
│   ├── book/
│   │   ├── handler.go
│   │   ├── service.go
│   │   ├── repository.go
│   │   └── model.go
│   ├── review/
│   │   ├── handler.go
│   │   ├── service.go
│   │   ├── repository.go
│   │   └── model.go
│   └── platform/
│       ├── database/
│       └── middleware/
└── go.mod
```

Feature-Based - Repository interface

```go
// internal/user/repository.go
package user

import "context"

type Repository interface {
    ByID(ctx context.Context, id int) (User, error)
    All(ctx context.Context) ([]User, error)
    Create(ctx context.Context, u User) error
}
```

Feature-Based - Service

```go
// internal/user/service.go
package user

import "context"

type Service struct {
    Repo Repository
}

func (s *Service) User(ctx context.Context, id int) (User, error) {
    return s.Repo.ByID(ctx, id)
}
```

Feature-Based - Handler (Go 1.22+)

```go
// internal/user/handler.go
package user

import (
    "encoding/json"
    "net/http"
)

type Handler struct {
    Service *Service
}

func (h *Handler) Routes() map[string]http.HandlerFunc {
    return map[string]http.HandlerFunc{
        "GET /users":      h.list,
        "GET /users/{id}": h.get,
        "POST /users":     h.create,
    }
}

func (h *Handler) get(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")   // Go 1.22+
    user, _ := h.Service.User(r.Context(), atoi(id))
    json.NewEncoder(w).Encode(user)
}
```

Feature-Based - Composition root

```go
// cmd/server/main.go
package main

import (
    "net/http"
    "github.com/example/internal/user"
    "github.com/example/internal/book"
    "github.com/example/internal/platform/database"
)

func main() {
    db := database.MustOpen("connection")

    userRepo := user.NewPostgresRepository(db)
    userSvc := user.NewService(userRepo)
    userHandler := user.NewHandler(userSvc)

    mux := http.NewServeMux()
    for pattern, handler := range userHandler.Routes() {
        mux.HandleFunc(pattern, handler)
    }

    // ....
    http.ListenAndServe(":8080", mux)
}
```

Feature-Based - When it works

- Medium-to-large services with multiple distinct features
- Teams where different developers own different features
- Each feature independently understandable
- Microservice candidates - each feature folder could become its own service

Feature-Based - When it breaks

- Cross feature concerns (e.g., "borrow book" touches user and book)
- Small applications - overhead of multiple packages
- Requires discipline to keep `platform/` clean

Feature-Based - 2026 perspective

"You want to isolate or enforce a boundary between the package code and the rest of your project."

- Alex Edwards, "11 Tips for Structuring Your Go Projects"

"A good mental model is: domain folders group behavior by feature rather than by layer."

- Community consensus (2024–2025)

Layout #7 Hexagonal / Ports & Adapters

Alistair Cockburn (2005), adapted for Go (2022–2026)
Also known as: Ports & Adapters

Related: Onion Architecture (Palermo 2008), Clean Architecture (R. Martin 2012)

> "Code pertaining to the 'inside' part should not leak into the 'outside' part."
>
> - Alistair Cockburn (2005)

In practice: the domain core has ZERO external dependencies. Everything else is an adapter.

Layout #7 Hexagonal / Ports & Adapters

Hexagonal - Why "hexagonal"?

The shape is a visual metaphor to break free from layered (top-to-bottom) thinking.

It allows the people doing the drawing to have room to insert ports and adapters.

It makes the **inside-outside** asymmetry obvious and leaves room for multiple ports.

Most apps have 2–4 ports (Cockburn has never encountered more than four).

Hexagonal - Directory structure

```
.
├── cmd/
│   └── server/
│       └── main.go              # Composition root
├── internal/
│   ├── core/
│   │   ├── domain/              # Entities & value objects
│   │   │   ├── book.go
│   │   │   ├── user.go
│   │   │   └── review.go
│   │   ├── port/                # Interfaces (contracts)
│   │   │   ├── book_repository.go
│   │   │   ├── user_repository.go
│   │   │   └── book_service.go  # use case (inbound port)
│   │   └── service/             # Business logic / use cases
│   │       ├── book_service.go
│   │       └── user_service.go
│   └── adapter/
│       ├── in/                  # Driving adapters (inbound)
│       │   └── http/
│       │       ├── book_handler.go
│       │       └── router.go
│       └── out/                 # Driven adapters (outbound)
│           ├── postgres/
│           │   ├── book_repository.go
│           │   └── user_repository.go
│           └── memory/          # In-memory for tests
│               ├── book_repository.go
```

Hexagonal - Domain core (zero dependencies)

```go
// internal/core/domain/book.go
package domain

type Book struct {
    ID       int
    Title    string
    Author   string
    Borrower *User
}
```

```go
// internal/core/port/book_repository.go - outbound port
package port

import (
    "context"
    "github.com/example/internal/core/domain"
)

type BookRepository interface {
    ByID(ctx context.Context, id int) (domain.Book, error)
    All(ctx context.Context) ([]domain.Book, error)
    Save(ctx context.Context, book domain.Book) error
    Update(ctx context.Context, book domain.Book) error
}
```

Hexagonal - Business logic

```go
// internal/core/service/book_service.go
package service

import (
    "context"
    "fmt"
    "github.com/example/internal/core/domain"
    "github.com/example/internal/core/port"
)

type BookService struct {
    books port.BookRepository
    users port.UserRepository
}

func (s *BookService) Borrow(ctx context.Context, bookID, userID int) error {
    book, err := s.books.ByID(ctx, bookID)
    if err != nil {
        return fmt.Errorf("find book: %w", err)
    }
    if book.Borrower != nil {
        return fmt.Errorf("book already borrowed")
    }

    user, err := s.users.ByID(ctx, userID)
    if err != nil {
        return fmt.Errorf("find user: %w", err)
    }
```

Hexagonal - Outbound adapter (PostgreSQL)

```go
// internal/adapter/out/postgres/book_repository.go
package postgres

import (
    "context"
    "database/sql"
    "github.com/example/internal/core/domain"
    "github.com/example/internal/core/port"
)

// Compile-time check: implements port.BookRepository
var _ port.BookRepository = (*BookRepository)(nil)

type BookRepository struct { db *sql.DB }

func (r *BookRepository) ByID(ctx context.Context, id int) (domain.Book, error) {
    var b domain.Book
    err := r.db.QueryRowContext(ctx,
        "SELECT id, title, author FROM books WHERE id = $1", id,
    ).Scan(&b.ID, &b.Title, &b.Author)
    return b, err
}
```

Hexagonal - Inbound adapter (HTTP)

```go
// internal/adapter/in/http/book_handler.go
package http

import (
    "encoding/json"
    "net/http"
    "github.com/example/internal/core/service"
)

type BookHandler struct {
    svc *service.BookService
}

func (h *BookHandler) Routes() map[string]http.HandlerFunc {
    return map[string]http.HandlerFunc{
        "GET /books/{id}":        h.get,
        "POST /books/{id}/borrow": h.borrow,
    }
}

func (h *BookHandler) get(w http.ResponseWriter, r *http.Request) {
    id, _ := strconv.Atoi(r.PathValue("id"))
    book, err := h.svc.Get(r.Context(), id)
    if err != nil {
        http.Error(w, "not found", 404)
        return
    }
    json.NewEncoder(w).Encode(book)
```

Hexagonal - Composition root

```go
// cmd/server/main.go - THE ONLY PLACE that knows about concrete types
package main

import (
    "log"
    "net/http"
    "database/sql"
    _ "github.com/lib/pq"

    "github.com/example/internal/adapter/in/http"
    "github.com/example/internal/adapter/out/postgres"
    "github.com/example/internal/core/service"
)

func main() {
    db, _ := sql.Open("postgres", "connection")

    // Outbound adapters (choose implementation)
    bookRepo := postgres.NewBookRepository(db)
    userRepo := postgres.NewUserRepository(db)
    // OR for testing: bookRepo := memory.NewBookRepository()

    // Core services (pure business logic)
    bookSvc := service.NewBookService(bookRepo, userRepo)

    // Inbound adapters
    bookHandler := http.NewBookHandler(bookSvc)
```

Hexagonal - The killer feature: testing

```go
// internal/adapter/out/memory/book_repository.go - test double
package memory

type BookRepository struct {
    books map[int]domain.Book
}

func (r *BookRepository) ByID(_ context.Context, id int) (domain.Book, error) {
    b, ok := r.books[id]
    if !ok { return domain.Book{}, fmt.Errorf("not found") }
    return b, nil
}
```

```go
// Test - no database needed!
func TestBorrowBook(t *testing.T) {
    bookRepo := memory.NewBookRepository()
    userRepo := memory.NewUserRepository()
    svc := service.NewBookService(bookRepo, userRepo)

    bookRepo.Save(context.Background(), domain.Book{ID: 1, Title: "Go 101"})
    err := svc.Borrow(context.Background(), 1, 42)
    assert.NoError(t, err)
}
```

Hexagonal - When it works

- Services with clear business rules protected from infrastructure changes
- Multiple implementations of the same port (Postgres, In-memory)
- Tests that need to replace some external systems
- APIs that might change transport (HTTP, gRPC)

Hexagonal - When it breaks

- CRUD-only services - abstraction overhead isn't worth it
- Small teams - extra indirection slows development
- Risk of "architecture astronaut" behavior
- Simple scripts and tools

Hexagonal - 2026 perspective

"Hexagonal architecture in Go is about keeping business logic pure while making infrastructure pluggable. It's not about folder names. It's about dependency direction."

- Community interpretation (2024–2025)

"Clean Architecture struggles in Golang because Go's philosophy favors simplicity over abstraction layers."

- Lucas de Ataides, dev.to (2024)

Layout #8 - Monorepo / Go Workspaces

Go 1.18 workspaces (2022), matured through Go 1.23–1.24

Monorepo - Directory structure

```
library-monorepo/
├── go.work
├── services/
│   ├── api/                    # HTTP API service
│   │   ├── go.mod
│   │   └── cmd/api/main.go
│   └── worker/                 # Background job worker
│       ├── go.mod
│       └── cmd/worker/main.go
├── libs/                       # Shared libraries
│   ├── library/                # Domain types & interfaces
│   │   ├── go.mod
│   │   ├── book.go
│   │   └── user.go
│   └── dbutil/                 # Database utilities
│       ├── go.mod
│       └── dbutil.go
└── tools/                      # Development tools
    └── migrate/
        ├── go.mod
        └── main.go
```

Monorepo - go.work

```
// go.work
go 1.24

use (
    ./services/api
    ./services/worker
    ./libs/library
    ./libs/dbutil
    ./tools/migrate
)
```

```
// services/api/cmd/api/main.go
package main

import (
    "github.com/example/libs/library"
    "github.com/example/libs/dbutil"
)

func main() {
    db := dbutil.MustOpen("connection")
    _ = library.Book{Title: "Go 101"}
    // ...
}
```

Monorepo - Two strategies

Strategy A: Multi-module (go.work)

- Each service and library has its own `go.mod`
- `go.work` ties them together for local dev
- Pros: Independent versioning, clear boundaries
- Cons: More setup, module management overhead

Monorepo - Two strategies

Strategy B: Single-module monorepo

- One `go.mod` at root, everything is a package
- Use `internal/` for boundaries
- Pros: Simpler, no module management
- Cons: Everything rebuilds together, no independent versioning

```
library-monorepo/
├── go.mod              # ONE module for everything
├── services/
│   ├── api/main.go     # package main
│   └── worker/main.go  # package main
├── internal/
│   ├── user/
│   ├── book/
│   └── review/
└── pkg/dbutil/
```

Monorepo - When it works / breaks

Works when:

- Multiple deployable services sharing domain logic
- Organization with 3+ Go services
- Different services have different release cycles

Breaks when:

- Single / Few services - monorepo adds complexity for no benefit
- Teams can't agree on shared library versions

Bonus - Go 1.22+ Standard Library Routing

Go 1.22 eliminated the need for 3rd-party routers.

After Go 1.22 - Standard library does it all

```
// Required: chi, gorilla/mux, echo, gin, fiber, etc.
r := chi.NewRouter()
r.Get("/users/{id}", getUser)
r.Post("/users", createUser)
```

Impact on structure: you needed a `router/` or `transport/` package
to encapsulate the third-party dependency.

```
mux := http.NewServeMux()
mux.HandleFunc("GET /users", listUsers)
mux.HandleFunc("GET /users/{id}", getUser)
mux.HandleFunc("POST /users", createUser)
mux.HandleFunc("GET /books/{path...}", getBook)  // wildcard

// In handlers:
func getUser(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")  // built-in path parameter access
}
```

Routing - Structural impact

- **No third-party router dependency** - one less package to organize
- **Method-based patterns** (`"GET /users"`) - handlers can be plain functions
- **Path parameters built-in** - no request context middleware needed
- **Handlers can live closer to domain** - fewer indirection layers

```
// internal/user/handler.go - clean, no external router dependency
package user

func Routes(svc *Service) map[string]http.HandlerFunc {
    return map[string]http.HandlerFunc{
        "GET /users":      List(svc),
        "GET /users/{id}": Get(svc),
        "POST /users":     Create(svc),
    }
```

Comparison - Side by side

Layout	Complexity	Best For
Flat	⭐	Prototypes, CLIs
Ben Johnson	⭐⭐	Small to medium services
William Kennedy	⭐⭐⭐	Enterprise services
Bourgon (pkg/)	⭐⭐	Monorepos (legacy)
Standard Layout	⭐⭐⭐⭐	Large complex projects
Feature-Based	⭐⭐⭐	Multi-feature services, DDD
Hexagonal	⭐⭐⭐⭐⭐	Enterprise services, DDD, Jav like

Universal principles

Regardless of layout choice (according to me!):

1. **Start simple** - add structure when you feel pain, not before
2. **Use** `cmd/` for entry points when you have multiple binaries
3. **Use** `internal/` if you **can't / won't** offer stable libraries
4. **Keep dependency arrows pointing inward** - domain shouldn't know about HTTP or Postgres
5. **Binary entry point e.g.** `main.go` **wires everything** - dependency injection at the composition root
6. **Organize by domain**, not by technical layer - `user/`, `book/`, not `handlers/`, `models/`

Key takeaways

1. **There is no One True Layout™** - Go gives you freedom, use it wisely
2. **The community has evolved** - from "copy this template" to "start simple, grow when needed"
3. **Dependency direction matters more than directory names** - domain logic should never import infrastructure
4. **The best layout is the one your team can maintain** - consistency beats perfection
5. **Go 1.22+ changes the game** - stdlib routing means fewer third-party deps and simpler structures

Older layouts resources

- Ben Johnson - Standard Package Layout
- William Kennedy - Package-Oriented Design
- Peter Bourgon - Go Best Practices

Newer layouts resources

- Alex Edwards - 11 Tips for Structuring Your Go Projects
- LaurentSV - No-Nonsense Go Package Layout
- Mat Ryer - How I Write HTTP Services After 13 Years
- Go Blog - Routing Enhancements
- Go Docs - Tutorial: Workspaces

Critical perspectives

- Waken Dev - Go Project Layout Controversy
- Kaushik Gopal - Cargo Culting
- Lucas de Ataides - Why Clean Architecture Struggles in Golang

Layout examples

Closing quote

"A good project structure is like good error handling, you only notice it when it's missing."

Thank you!

Thank you! 😊

Done with github.com/oktalz/present